

10/03/2026

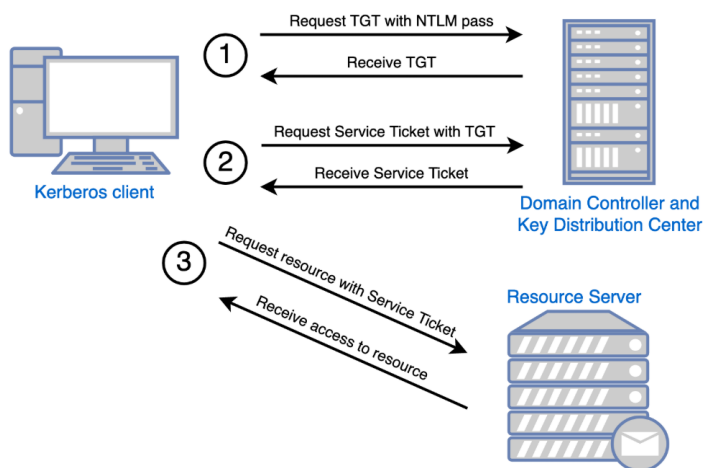
Single sign-On, vogliamo una password unica per accedere a tutto il sistema. Quando accediamo ad un sistema e ci autenticiamo non è detto che tutti possano accedere a tutto. Abbiamo bisogno sia di autenticazione sia di autorizzazione sui sistemi. L'autorizzazione dipende in maniera stretta dall'autenticazione. e.

Kerberos riesce a realizzare questo tipo di soluzione.

Kerberos nacque nella fine degli anni 80 come evoluzione di symmetric needham schroder che sostituisce la nonce con i timestamp. All'inizio i timestamp non erano utilizzati poiché richiedevano che **le macchine fossero sincronizzate** (cosa che all'epoca non era così). Quindi, vedendo un messaggio con un certo timestamp, posso decidere se il tempo è minore o maggiore rispetto a quello che io sono disposto ad accettare. Il timestamp però non ci garantisce il fatto che sia veritiero, non è detto che chi lo abbia messo non ne abbia messo uno futuro per farlo accettare. Ci fideremo dei Timestamp solo se generati da un TTP o comunque dovremmo presumere possa essere stata falsata.

Gli obiettivi di Kerberos sono segretezza, autenticazione ad accesso singolo e temporalità che ci permette di prevenire replay.

La struttura base di Kerberos è questa:



1. Autenticazione: Il client si autentica con l'autentication server. (AS)
2. Autorizzazione: Il client chiede un ticket al Ticket-granting Server. (TGS) (opzionale)
3. Accesso al servizio: Il client fornisce il ticket al servizio a cui vuole accedere. (opzionale)

2 e 3 sono opzionali perché non è detto che il client voglia accedervi. Il tutto è trasparente al client.

L'autentication server fornisce un **AuthKey** (chiave di sessione) e un **AuthTicket** (ticket per accedere al TGS per autenticarsi nel sistema). Allo stesso modo la fase 2 fornisce **ServKey** e **ServTicket**. Le credenziali di autenticazione in genere hanno valenza di circa 8 ore (giornata

lavorativa) mentre per le credenziali di servizio si parla di pochi secondi perché devono essere fresche. Una AuthKey può criptare diverse servKey cioè un utente autorizzato può avere più chiavi di accesso ai servizi.

Kerberos funziona come segue:

• I. AUTENTICAZIONE

1. $A \rightarrow AS : A, TGS, T_1$ *metto un nuovo Timestamp perché non ci fidiamo!*

2. $AS \rightarrow A : \{authK, TGS, Ta, \{A, TGS, authK, Ta\}K_{TGS}\}K_A$

• II. AUTORIZZAZIONE

3. $A \rightarrow TGS : \{A, TGS, authK, Ta\}K_{TGS}, \{A, T_2\}authK, B$

4. $TGS \rightarrow A : \{servK, B, Ts, \{A, B, servK, Ts\}K_B\}authK$

• III. SERVIZIO

5. $A \rightarrow B : \{A, B, servK, Ts\}K_B, \{A, T_3\}servK$

6. $B \rightarrow A : \{T_3+1\}servK$

le due componenti non sono considerate

richiedo il servizio di comunicare con B

utile per prevenire replay; devono essere tutte fresche!

chiave di sessione tra A e B

esso ci farebbe anche questo volendo

T_1, T_2, T_3 non sono indicatori sicuri. Sono più affidabili perché cifrati. Danno una indicazione temporale all'interlocutore

C'è una dipendenza tra la chiave authK e la chiave servK al messaggio 4 perché chiunque conosce authK può conoscere servK. Proprio a causa di ciò, vogliamo garantire che il tutto è fresco proprio grazie ai Timestamp.

L'attacco a Needham Schroder derivava da Dolev-Yao + che si può impossessare di qualche vecchia chiave di sessione. Da qui consideriamo il Lifetime della chiave.

Cosa succede se un attaccante si prende il messaggio 2 e 4 e se li salva? In needham schroder non ha senso parlare di sessioni scadute perché ci sono le nonce e qualunque cosa che corrisponde alle nonce diciamo che è okay.

A questo punto però bisogna chiedersi, cosa succede quando scade una sessione? Che fine fanno le vecchie chiavi di sessione? È naturale che le servkey ottenute da una specifica authKey, debbano essere eliminate una volta che l'authkey scade (esempio, l'utente viene killato). Il problema è che la servicekey potrebbe durare più di quanto dura authkey.

Cosa potrebbe accadere che con l'authkey presa dall'attaccante, questo possa impadronirsi di una servkey che è ancora valida nonostante l'authkey non lo sia più.

Per evitare questo problema, diciamo che non produciamo più servkey se il lifetime di queste sfiorerebbe la durata dell'authkey oppure facciamo sì che la servkey scada quando scade l'authkey.

12/03/2026

Dal momento che $T_s + L_s \leq T_a + L_a$ B può fidarsi

Cascade Attack, non si può fare se imponiamo quella relazione poiché l'attaccante non si può procurare una servK valida.

Vecchio è generico, scaduto è preciso.

Kerberos 4

No realm. Limiti ambientali e tecnici.

Doppia codifica: il TGS ha conferma che anche A ha ricevuto l'authkey. Per via dell'identificatore l'inoltro preventivo non funzionerebbe.

Kerberos 5 la elimina la cifratura annidata perché con l'autenticatore non serve.

Otway protocol approfondimento

Password Kerberos 5

Smart Card

La smart card esegue la crittografia ma quanto è forte la smart card? I messaggi pari vengono decifrati dalla smartcard. Dalla smartcard non esce mai la chiave reale, esce solo la chiave di sessione. Questa cosa viene adesso fatto negli NFC. Java Card è un mini computer programmabile.

TGT (authTicket)

Smart Card il chip esegue il tutto come un mini computer (non ci sarà mai una chiamata che gli permette di dare i suoi dati). Carte magnetiche ha solo dati.

E come si scrive su una smart Card

17/03/2026

Il pin in una smart card come è protetto? L'utente si autentica tramite il PIN alla carta. Come si salva il pin in una smart card?

Il pin viene mandato inizialmente in chiaro a un microprocessore fidato che lo cifra. Facciamo sì che quindi questo flusso sia estremamente breve e difficilmente intercettabile.

Per cercare di fare questo genere di cose, sarebbe necessario dunque rubare il pin PRIMA che questo venga inviato.

Perché usiamo ancora PIN e carta al bancomat se esistono le app con OTP ecc.

La smartcard è un microcomputer a tutti gli effetti. Fisicamente quanto è robusta una carta di questo tipo ad attacchi fisici?

Le vecchie sim avevano una rom che conteneva i numeri.

Standard ISO 7810

Attaccanti

Per quanto la chiave non esca mai dalla carta, questa deve prima o poi andare in CPU. Per cui si può fare l'attacco inserendo un intercettatore tra rom e cpu.

Eavesdropping: ascoltiamo le radiazioni elettromagnetiche per intercettare i dati

Fault induction.

If(a == 0) goto 8 perché è fondamentale? Cosa succede con un leggero cambio di tensione (fault induction)

Crittografia Visuale

Lo XOR è invertibile mentre l'OR no.

È più simile all'OR che allo XOR

19/03/2026

Esposito Chan

Red team vs Blue team. La sicurezza è un processo che non si può mai ottenere.

PvE.

Definizione vulnerabilità: violazione di Confidenzialità, disponibilità integrità. (CIA)

CVE mroto o quasi.

CWE

Exploit

Bind shell è la vittima che espone la shell (ssh). Firewall fa na pernacchia

Rev Shell è la vittima che "manda" la shell sul listener dell'attaccante

Metasploit

Serve principalmente a sfruttare vulnerabilità.

Tutto si basa su moduli

24/03/2026

Encryption simmetrica. Idea One-Time Pad con XOR. Lo xor è robusto solo se la chiave ha la stessa lunghezza del messaggio e se viene usata solo una volta. Tabella di verità per overlay.

Inizialmente si fa la prova con singoli pixel trasparenti ci si rende conto che non funziona perché non è proprio la classica tabella di XOR. Considerando matrici due per due, di base non sarebbe simmetrica alla classica tabella dello XOR, ma la tabella che facciamo a livello logico per sovrapposizione di pixel è uguale poiché il nostro occhio non riesce a distinguere i vari casi.

Requisito due pixel neri e due bianchi (verticale, orizzontale, diagonale). Li dobbiamo scegliere in maniera tale che per sovrapposizione o risultano due pixel neri e due bianchi o tutti neri.

Algoritmo di codifica:

1. Per ogni pixel dell'immagine, scegliere un bit a caso e associargli matrice secondo lucido 13. La scelta randomica è necessaria al fine di utilizzare one time pad. (creazione della chiave)

2. Per ogni pixel nero dell'immagine di partenza, scegliere la matrice complementare della chiave del pixel, altrimenti la stessa matrice. Così facendo, la sovrapposizione dei due share darà l'immagine corretta.

Lo sfondo sarà grigiastro perché il nostro occhio funziona così. Stacce.

Aggiungiamo più bit per ottenere più scale di grigio. Nel caso di 4 livelli di grigio, abbiamo bisogno di una matrice 3×3 .

Perché abbiamo bisogno di 2 neri e 2 bianchi poiché altrimenti avremmo una perdita di informazioni.

Abbiamo bisogno che siano uguali le sovrapposizioni nel caso della non presenza di informazioni perché altrimenti il nostro occhio non li percepirebbe uguali e non riuscirebbe a distinguere lo "sfondo". Non ci sarebbe abbastanza contrasto.

Motivo perché non funziona lo schema $3 + 1$ o meglio non potrebbe funzionare. Abbiamo bisogno che per sovrapposizione si ottenga un certo numero di pixel neri o un po' più o un po' di più e così via...

Con uno schema 3×3 posso riuscire a generare più incroci e ottenere così 4 scale di grigio diversi.

Da 2×2 posso anche ottenere 3 scale di grigi anche se non si distinguono molto. Same per 3×3 che potresti ottenere fino a 5 livelli di grigio. Più ci avviciniamo al limite di pixel accensibili, più non si capisce perché il nostro occhio fa più fatica.

È importante che tutte le matrici siano equiprobabili così che qualunque riesca ad intercettare non avrebbe modo di riuscire a fare guess sensate.

Vogliamo massimizzare il contrasto e garantire l'equiprobabilità.

Il primo lo prendo per convenzione il secondo lo posso scegliere io.

- Schemi a soglia (Threshold secret sharing): visuali per cui servono almeno k persone per decifrare.
- Generale access secret sharing: solo specifici gruppi.

EVC che c'è nelle slide è del secondo tipo.

Non ripudio

Come rendere una proprietà di questo genere? È importante in un mondo digitale di questo tipo.

La firma digitale funziona tramite crittografia asimmetrica. Abbiamo bisogno di un certificato che certifichi la nostra chiave pubblica.

La non ripudiabilità prima si faceva con la firma reale. Anche in informatica allora la firma digitale garantisce la non ripudiabilità.

L'algoritmo di verifica della firma fa parte della firma digitale stessa in un certo senso. Altrimenti non avrebbe senso.

I problemi nascono dai dettagli.

Delega, come la facciamo?

Mutualità del non ripudio non è banale. Uno dei due può sempre barare. (A e B non si fidano più tra di loro, altrimenti la soluzione sarebbe banale modello general attacker). Un altro problema è il mezzo di comunicazione (?)

Amazon risolve il problema del non ripudio nel modo più semplice, ti manda dei pacchi.

Non ripudio:

Disponibilità di evidenza inequivocabile che impedisca a un soggetto di negare proprie azioni

Vedremo:

1. Compravendita
2. Delega
3. E-mail

Problema simile ma ambienti diversi.

Fattori determinanti:

1. Affidabilità del mezzo.
2. Onestà dei partecipanti.

26/03/2026

Passiamo da Dolev-Yao a General attacker poiché adesso non è detto che i due agenti A e B si fidino tra loro.

Siamo in un sistema a chiave asimmetrica e l'oggetto del desiderio è il messaggio m .
Vogliamo non ripudio di spedizione e di ricezione.

Il non ripudio in questo contesto è definito come Equa compravendita.

'Equa compravendita

Def. A spedisca a B un messaggio m in modo tale che A ottenga evidenza che B l'abbia ricevuto se e solo se B ottenga evidenza che A l'abbia spedito

Cerchiamo di risolvere il problema in un caso ovvio:

1. $A \rightarrow B : \text{flag_nro}, B, m, \text{Sign}_a(\text{flag_nro}, B, m)$
2. $B \rightarrow A : \text{flag_nrr}, A, \text{Sign}_b(\text{flag_nrr}, A, m)$

F_{nro} ("Non-Repudiation of Origin") f_{nrr} ("Non-Repudiation of Receipt")

Vediamo adesso il caso in cui ci sia un mezzo affidabile ma agenti disonesti:

1. $A \rightarrow \text{TTP} : f_{nro}, \text{TTP}, B, m, \text{Sign}_A(f_{nro}, \text{TTP}, B, m)$
2. $\text{TTP} \rightarrow B : f_{nrs}, A, B, m, \text{Sign}_{\text{TTP}}(f_{nrs}, A, B, m)$
3. $\text{TTP} \rightarrow A : f_{nrd}, A, B, \text{Sign}_{\text{TTP}}(f_{nrd}, A, B, m)$

F_{nrs} submission f_{nrd} delivery

Terzo caso, mezzo inaffidabile e agenti onesti:

1. $A \rightarrow B : f_{nro}, B, m, \text{Sign}_A(f_{nro}, B, m)$
2. $B \rightarrow A : f_{nrr}, A, \text{Sign}_B(f_{nrr}, A, m)$
3. $A \rightarrow B : f_{ack}, B, \text{Sign}_A(f_{ack}, B, m)$

Mando finché non ricevo l'ack. Se il messaggio 1 non arriva, non ci sono dispute, la proprietà non è violata. Per questo motivo non c'è l'ack nel primo messaggio. Sul due c'è l'ack perché se non arriva il protocollo non è andato a buon fine e B riceve la conferma d'invio senza dare la conferma di ricezione.

Rilascio posticipato con agenti e canale inaffidabili:

- Inizialmente dare il crittotesto al ricevente
- Se il ricevente è interessato a decodificare il crittotesto deve continuare la sessione
- Mentre continua dà evidenza di partecipazione
- Infine dare al ricevente la chiave

Mandare il crittotesto rispetta la proprietà perché comunque non c'è conferma di nulla.

Poe promise of exchange

1. $A \longrightarrow B : f_{poe}, B, c, \text{Sign}_A(f_{poe}, B, c)$
2. $B \longrightarrow A : f_{acp}, A, \text{Sign}_B(f_{acp}, A, c)$
3. $A \longrightarrow B : f_{nro}, B, k, \text{Sign}_A(f_{nro}, B, k)$
4. $B \longrightarrow A : f_{nrr}, A, \text{Sign}_B(f_{nrr}, A, k)$

B può scappare...

Ci sono 3 strategie:

1. TTP
2. Reiterazione
3. Rilascio posticipato

3 strategie su 2 problemi (????)

Nel terzo caso non hanno usato le prime due soluzioni.

Nel loro protocollo usano il TTP e un qualcos'altro al posto della reiterazione.

Supponiamo che il mezzo sia NON SEMPRE AFFIDABILE altrimenti la reiterazione non basta.

Se il mezzo non è affidabile sempre, prima o poi funziona e quindi se non riesci a prenderti il messaggio è un tuo problema.

Il ttp lo rende disponibile (FTP) , quindi è un tuo problema.

A cosa serve il rilascio posticipato allora?

Label è una sorta di nonce e serve da perno per risolvere le dispute. La label e crittotesto sono legati e quindi sa che k è la chiave per decifrare c .

I primi messaggi sono una sorta di manifestazione d'intenti.

È valido questo protocollo a smascherare ognuno la menzogna dell'altro.

Il messaggio è confidenziale? Se non serve, allora perché il rilascio posticipato ha il crittotesto?

Versione senza cifratura e senza rilascio posticipato.

Il TTP deve essere stateful per evitare replay entro certi limiti.

Non ripudio in E-mail (PEC)

C'è uno che dice che io ho firmato. Mi sto fidando.

La pec è un protocollo di Trustness. La firma la fa solo il gestore e garantisce solo che lui abbia fatto il suo dovere.

La pec è un sistema tutto italiano.

PKI: scambio di chiave tra persone che non si conoscono. Problemi di scalabilità.

In teoria non c'è il TTP però tutti condividono una chiave condivisa con il TTP.

07/04/2026

Protocollo Abadi et al.

Premesse tecniche:

- Mittente S, ricevente R, e-mail m
- Stateless TTP
- No PKI ma ogni ente ha una chiave condivisa con il TTP (questa cosa non è scalabile, ergo, servirebbe PKI)
- S e R a coppia hanno una funzione di **ack(q) = r** cioè si autenticano tra di loro con una certa funzione. (c'è un segreto tra s e r a gratis) (No pki e S e R che hanno funzioni condivise è l'equivalente di una pki) Perché usare una funzione anziché un semplice segreto? La dichiarazione della funzione è "costante", cioè allo stesso x corrisponderà sempre lo stesso y. La legge è costante. Ack ha aereità 1 (?). Una costante è una nullary function (non prende parametri). Anziché avere una chiave costante (una funzione ad aereità 0) abbiamo una legge costante. (cambia poco dall'avere una chiave costante)
- SSL

Tramite queste premesse, vogliamo ottenere un equo recapito. In forma debole:

Def. Il ricevente legga l'e-mail se e solo se il mittente riceva la ricevuta di ritorno

Il protocollo della raccomandata rr è attaccabile banalmente il mittente non è autenticato. Allo stesso modo la proprietà di equo recapito debole è attaccabile ma di base la proprietà non è violata.

Versione forte:

Def. Il ricevente legga l'e-mail, ottenendo pure evidenza che provenga dal mittente, se e solo se il mittente riceva la ricevuta di ritorno

Prima la ricevuta non era equa perché la ricevuta agli altri era molto più forte rispetto alla semplice e-mail ricevuta dal destinatario. Il mittente ha la mail e la ricevuta di ritorno e il destinatario ha la mail e la ricevuta dell'autenticazione del messaggio.

La PEC è basata su trust nell'entità centrale. La sicurezza sarebbe sempre garantita se buttassimo TTP ovunque. Il nostro obiettivo è quello di ridurlo il più possibile.

■ Abbreviazioni

$$\begin{aligned}c &= m_k \\h_S &= \text{Hash}(q, r, c) \\h_R &= \text{Hash}(q', r', c') \\S2TTP &= \{S, k, R, h_S\}_{\text{pubEK}_{TTP}} \\DR &= \text{Sign}_{TTP} S2TTP\end{aligned}$$

c è il messaggio cifrato. $S2TTP$ è un messaggio firmato con la chiave pubblica del destinatario (cioè il TTP) questo vuol dire che è un messaggio non autenticato. La ricevuta di ritorno è sempre il $S2TTP$ per firmato con il TTP. H_r e H_s sono degli hash fatti da s e r e la funzione hash la conoscono solo loro. Notiamo che gli hash sono con q e r e con q' e r' . È chiaro che vogliamo qui una base di sicurezza (su SSL) e, dunque, sappiamo che c'è Dolev Yao.

Guardiamo la riga 1 e notiamo che q arriva e potrebbe arrivare anche qualcos'altro dato che passa in chiaro. q potrebbe arrivare come q' a causa di un attaccante come q potrebbe essere uguale a q' nel caso in cui nessuno intervenga. Allo stesso modo c' potrebbe essere diverso da c . H_r e H_s o sono uguali o non lo sono.

Se sono uguali:

1. La funzione ack è per forza quella giusta
2. La rete non ha modificato nulla

Se sono diversi non è posso distinguere tra i due casi. I parametri sono identici e la r corrisponde alla specifica q che è stata costruita secondo la regola. Il TTP si rende conto che i due si sono autenticati se e solo se i due hash sono uguali. Il TTP siamo tranquillamente noi che possiamo vedere i due hash. Passiamo dunque ad una proprietà di primo livello.

1. $S \longrightarrow R : TTP, c, q, S2TTP$
2. $R \xrightarrow{SSL} TTP : S2TTP, \text{pwd } R, h_R$
3. $TTP \xrightarrow{SSL} R : k, h_R$
4. $TTP \longrightarrow S : DR$

Un forgery dell'attacco 1 è assolutamente fattibile ma poi non andrebbe a buon fine l'autenticazione a causa del fatto che non si conosce la funzione ack.

La funzione di ack serve a generare la response r. R proverà la funzione ack che condivide con S e, se non è S il vero sender (ma magari E), allora la r generata da E non corrisponde.

Scarico il problema sul TTP al messaggio 2. È il TTP a fare il confronto tra H_r e H_s e, se sono diversi, il TTP fa abort. S non si è dovuto autenticare esplicitamente con il TTP. R deduce che si è autenticato con S perché il TTP gli manda quell'hash che solo S poteva produrre. È il TTP che fa il confronto e mi dà la conferma.

H_r funge anche da label e sarà diverso per ogni crittotesto.

Il mittente ha la ricevuta di ritorno DR (proprietà di equa recapito debole) e il ricevente ha certezza che il messaggio è stato mandato da S perché l'equità di H_r e H_s autentica. Ma questo basta per ottenere recapito forte? Dovrei spiegare tutto il protocollo ecc. non è realmente dimostrabile al mondo l'autenticità di S. Il controllo non dovrebbe poterlo fare solo R ma anche tutto il mondo. Non c'è un'evidenza diretta dell'autenticazione di S. Il TTP confronta solo due hash.

L'unico a poter produrre una prova reale dell'autenticazione di S è il TTP che però non ha mai visto la sua password cioè non lo ha mai autenticato. Quindi possiamo mettere la password di S in $S2TTP$ così da poter fare l'autenticazione. Al messaggio 3 il TTP menziona S.

Manca il principio di explicitness all'inizio. Nel modo che abbiamo visto prima, il controllo viene fatto dopo, se manca la password all'inizio, l'abort avviene subito.

Non è il TTP che evita attacchi di Replay

La mancanza di PKI è tamponata da SSL.

L'equo recapito debole è più debole rispetto all'equa compravendita perché manca la proprietà di secondo livello.

Non ripudio della delega

La delega è un processo in cui si trasferisce a qualcuno un'autorizzazione a fare qualcosa (non un obbligo).

Consideriamo le seguenti precondizioni:

- Grantor G, grantee g
- Compito omega da fare presso un end-point (la delega non è un obbligo)
- Chiavi di firma certificate per G e g (PKI)
- Altre 3 coppie di chiavi asimmetriche, 1 da G e 2 da g.
- No TTP.

Non abbiamo più due player ma 3 dato che abbiamo anche l'end-point.

Abbiamo più coppie di chiavi per più obiettivi diversi (una coppia per la firma, una per la cifratura, una per esercitare la delega ecc.)

Vorremmo poter generare o accertare delle coppie di chiavi a partire da una master. La master è la mia o del grantor che, però, è certificata. Qualunque PKI può essere allungata a patto che io sia certificato e quelle che ho allungato hanno senso in funzione della mia chiave che è certificata.

Vogliamo ottenere equa delega:

Def. Il grantee riceva evidenza che il grantor gli ha dato diritto su Ω se e solo se il grantor riceva evidenza che il grantee ha accettato responsabilità su Ω

Vediamo le chiavi in gioco:

Compito	Sintassi		Algoritmo	Usata da	Certificazione
Firma	priSK	pubSK	firma	G e g	Esterna
Delega	priDK	pubDK	firma	G	Interna
Accettazione	priAK	pubAK	firma	g	Interna
Esercizio	priEK	pubEK	Codifica asim.	g	Interna

Tutte le chiavi a certificazione interna, sono certificate da quelle di firma. Vediamo il protocollo:

1. $G \longrightarrow g : \text{Sign}_{\text{priSK}_G}(G, g, \Omega, \text{pubDK}_G)$
2. $g \longrightarrow G : \text{Sign}_{\text{priSK}_g}(g, \Omega, \text{pubAK}_g, \text{pubEK}_g)$
3. $G \longrightarrow g : \text{Sign}_{\text{priDK}_G}(g, G, \Omega, \text{pubDK}_G, \text{pubEK}_g, \text{pubAK}_g)$

Ricevuto il messaggio 4 il grantee firma il messaggio 3 con la sua chiave privata di accettazione generando il token di delega:

$$\text{Sign}_{\text{priAK } g}(\text{Sign}_{\text{priDK } G}(g, G, \Omega, \text{pubDK } G, \text{pubEK } g, \text{pubAK } g))$$

Che verrà poi inviato all'end-point. L'end point dovrà controllare tutte le chiavi.

Se g chiude prima non ottiene modo di esercitare Ω . Ma se ottiene la possibilità di esercitare Ω , allora diventa anche responsabile. La responsabilità è nel bene o nel male.

G è in un lieve vantaggio perché ottiene 2? No perché l'end point non accetta se non ottiene il token.

■ Inerenti diritto/responsabilità

- ☐ Se G nega di avere delegato a g diritto su Ω , allora a g basta esibire il token di delega
- ☐ Se g nega di aver accettato da G responsabilità su Ω , allora a G basta esibire il messaggio 2

■ Inerenti l'esercizio di Ω

- ☐ Se g nega di aver esercitato Ω per conto di G , allora G deve esibire il messaggio 2 **insieme** al token di delega richiesto all'end-point

09/04/2026

Le chiavi sono certificate internamente.

Tutte queste chiavi non sembrano così tanto giustificate, ne basterebbero praticamente solo una per G e g .

Analisi formale dei protocolli di sicurezza

Problema semi decidibile (terminazione di un algoritmo)

L'analisi informale diciamo a parole non basta a dimostrare la sicurezza di un protocollo.

Quando il linguaggio informale trova un attacco l'attacco c'è, ma quando dobbiamo dire che un protocollo è sicuro la cosa diventa problematica.

Le dimostrazioni sono fatte per dimostrare fatti universali la sicurezza lo è?

Vogliamo traslare i teoremi matematici ai protocolli di sicurezza.

Model checking nasce negli anni 80. Il model checking serve a dimostrare delle proprietà in un modello finito e ben definito.

Una dimostrazione classica è una derivazione formale a partire da assiomi che valida tutti i casi possibili esistenti indipendentemente da enumerazioni.

Il model checking, invece, esplora tutti gli stati possibili in un modello finito. È un banale processo enumerativo. La state enumeration ha senso in funzione del modello finito.

Il modello, essendo limitato, è decidibile. Ma se trovo una proprietà dentro il modello limitato, allora posso generalizzarlo (lenticchia rossa in un barattolo, se la trovo posso generalizzare e dire che esistono lenticchie rosse al mondo. Se non la trovo, non posso dedurre nulla)

Il model checking serve a trovare attacchi non a dimostrare la sicurezza di un protocollo. Se trovassi una dimostrazione matematica, allora potrei utilizzarla per generalizzarla.

Validazione vs verifica (con la verifica la trovo la proprietà, con la validazione vedo che funziona)

L'analisi formale fa uso di model checking dagli anni 80 e le dimostrazioni matematiche da tipo sempre.

Algebra di processo (process calculus).

Abbiamo bisogno di un modello formale o una specifica (ccs, ban logic ecc.)

Ho la specifica, ho la proprietà, applico la proprietà alla specifica e faccio la verifica.

Descrivo il protocollo in un linguaggio (la specifica)

Poi specifico la proprietà. LTL modello di logica sul tempo. Non è detto che il linguaggio sia uguale. Nel modello induttivo si usa lo stesso.

Esistono dei tool per fare il model checking prendendo in input il protocollo + proprietà (negli specifici linguaggi). Enumera tutti gli stati e poi ci dice se c'è l'attacco. Potrebbe sembrare non finire mai (state explosion) a causa dell'elevato numero di stati esistenti.

Generalizzare è spesso impossibile.

Vogliamo ridurre i protocolli di sicurezza a delle istantanee dell'esecuzione di un protocollo. Cercheremo di rappresentare la conoscenza cioè è epistemologico.

Ma come ci arriviamo dalla specifica agli stati

Le formule sono un modello della realtà.

Per fare dimostrazioni per induzione abbiamo bisogno di un modello definito per induzione. Principio di induzione (atto di fede)

I numeri naturali sono generati per induzione (magia). Dal vero non posso dedurre il falso

Backward reasoning.

Devo dimostrare le precondizioni valide per il principio di induzione.

14/04/2026

La Fisica è una modellazione della realtà.

Sorgenti di infinitezza:

1. Quanti messaggi?
2. Quanto lunghi?
3. Quante chiavi da usare?
4. Quanti agenti eseguono il protocollo?
5. Quanto è potente l'attaccante?

Le dimostrazioni si fanno sul generico quindi sull'infinitezza. I protocolli di sicurezza in quanto distribuiti è difficile rappresentarli tramite macchine a stati finiti poiché proprio infiniti.

Ma quindi come lo facciamo il modello?

Proof assistants

È un tool software che permette di verificare una dimostrazione matematica o comunque di aiutarne la costruzione. Ci danno la possibilità di verificare le dimostrazioni già esistenti. Servono perché vogliamo prendere i nostri protocolli e verificarne la correttezza tramite vere e proprie dimostrazioni matematiche.

Higher Order logic

È la logica con cui il nostro protocollo è modello formalmente. È l'evoluzione della logica proposizionale

Logica del primo ordine è una logica che unisce alla logica proposizionale dei quantificatori per esprimere proprietà sulle variabili.

Logica del secondo ordine come prima ma i quantificatori possono valere anche per le proprietà.

Metodo induttivo

Un insieme induttivo è un insieme infinito che ha una serie di regole. Non lo descriviamo tramite enumerazione ma tramite le regole che lo definiscono. Quasi sempre abbiamo un caso base e una serie di proprietà per induzione.

Il metodo induttivo è un metodo che ci aiuta a dimostrare delle proprietà su un insieme induttivo senza dover verificare che la proprietà valga singolarmente per ogni elemento.

L'idea del metodo:

vedi slide

Isabelle

È un proof assistant che usiamo molto. Fa uso di Higher Order logic. Questo tool fa automated reasonings automatizzando molti passaggi.

Abbiamo delle **teorie** e questi contengono dei lemmi, definizioni ecc. Sono delle librerie che carichiamo e che ci aiuteranno a comporre le nostre dimostrazioni. Nelle teorie le cose sono già dimostrate.

Isabelle è molto usato perché ci aiuta a modellare i protocolli di sicurezza e verificare delle proprietà di sicurezza dello stesso.

Quando abbiamo un protocollo, questo lo vediamo come una lista di eventi/messaggi. Possiamo vedere il nostro protocollo come una sorta di induzione specificando il caso base e poi facendo sì che le regole estendano il caso base con nuovi eventi.

Gli eventi sono praticamente delle regole. Il protocollo è composto da tutte le regole che lo compongono che sono tutte le possibili combinazioni di regole.

Nil caso base il resto sono passi induttivi ma ce n'è più di uno. Questo insieme è chiuso sotto le regole date e da questo posso derivare solo elementi nell'insieme.

Il caso base è quello che non ha precondizioni.

Gli elementi del nostro insieme sono liste composte da regole del nostro insieme (da tutte le possibili combinazioni di regole del nostro insieme)

[Says AB] è un elemento valido del protocollo [Says A B, Says BA] è anche un elemento valido del protocollo. Modelliamo cosa significa eseguire un protocollo.

Esempio:

[]

[Says A B {A, Na}]

[Says A B {A,Na}, Gets B Message]

Poiché le regole hanno delle precondizioni, non possiamo eseguirle a caso.

Un evento è un'azione che può accadere. Una traccia è una lista valida del nostro protocollo. Ogni possibile esecuzione del nostro protocollo è una traccia e sono infinite. L'attaccante è anch'esso sviluppato tramite una regola

Gli eventi sono le azioni che abbiamo scelto di voler tenere in considerazione. La lista è una scelta semplice per descrivere l'esecuzione del protocollo. Abbiamo l'insieme di tutte le possibili tracce cioè di tutte le tipiche esecuzioni del protocollo (comprese le più strane o compresi gli attacchi). Poniamo il focus su quello che ci interessa.

Definizione delle chiavi simmetriche tramite notazione Isabelle e agents

Esempi di tracce

Parts prende una traccia del protocollo ed estrae dei messaggi.